

LAPORAN PRATIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 5”

Disusun oleh:

Rifal maulana oskar

2511533024

Dosen Pengampu : Dr. Wahyudi, S.T, M.T

Asisten Praktikum : Sherly sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum ini dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum struktur data pada pertemuan keempat dengan materi dasar **NodeSLL**.

Semoga laporan ini dapat memberikan pemahaman dasar mengenai penggunaan bahasa pemrograman Java, khususnya dalam mengenal struktur data.

Penulis menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Padang, 07 mei 2026

Rifal maulana oskar

DAFTAR ISI

BAB I	4
1.1 Latar belakang.	4
1.2 Tujuan.	4
1.3 Manfaat.	5
BAB II	6
2.1 Pengertian SLL.	6
2.2 Struktur Node dalam SLL.	6
2.3 Operasi penambahan Node (insert) pada SLL.	7
2.4 Operasi penghapusan Node (delete) pada SLL	8
BAB III	10
DAFTAR PUSTAKA	11

BAB I

PENDAHULUAN

1.1 Latar belakang.

Struktur data merupakan salah satu konsep penting dalam ilmu komputer yang digunakan untuk menyimpan, mengelola, serta memanipulasi data agar dapat diakses secara efisien. Pemilihan struktur data yang tepat sangat berpengaruh terhadap kinerja suatu program, terutama ketika program menangani data dalam jumlah besar. Salah satu struktur data yang sering digunakan dalam pemrograman adalah Linked List.

Linked List adalah struktur data dinamis yang terdiri dari sekumpulan node atau simpul yang saling terhubung menggunakan pointer atau referensi. Berbeda dengan array yang memiliki ukuran tetap, linked list dapat bertambah dan berkurang sesuai kebutuhan selama program berjalan. Hal ini membuat linked list lebih fleksibel dibandingkan array, terutama dalam kasus penambahan dan penghapusan data.

Single Linked List (SLL) merupakan jenis linked list yang paling sederhana, dimana setiap node hanya memiliki satu pointer yang menunjuk ke node berikutnya. Single Linked List banyak digunakan dalam implementasi sistem antrian, stack, serta pengelolaan data yang membutuhkan perubahan ukuran secara dinamis. Oleh karena itu, pemahaman tentang operasi dasar Single Linked List seperti penambahan node, penghapusan node, dan pencarian node sangat penting untuk dipelajari dalam mata kuliah Struktur Data.

Dalam makalah ini akan dibahas implementasi Single Linked List menggunakan bahasa pemrograman Java, yang mencakup pembuatan node, proses penambahan node di depan, di belakang, dan di posisi tertentu, proses penghapusan node pada head, tail, dan posisi tertentu, serta proses pencarian data pada linked list.

1.2 Tujuan.

Tujuan dari penulisan makalah ini adalah:

- ☐ Menjelaskan konsep dasar Single Linked List.
- ☐ Menjelaskan struktur node dalam implementasi Single Linked List.
- ☐ Menjelaskan cara menambahkan node pada Single Linked List menggunakan Java.
- ☐ Menjelaskan cara menghapus node pada Single Linked List menggunakan Java.
- ☐ Menjelaskan cara mencari data pada Single Linked List menggunakan Java..

1.3 Manfaat.

- ☐ Menambah wawasan mengenai konsep dan implementasi Single Linked List.
- ☐ Membantu memahami operasi dasar linked list seperti insert, delete, traversal, dan search.
- ☐ Memberikan contoh implementasi Single Linked List menggunakan bahasa Java yang dapat dipelajari serta dikembangkan lebih lanjut.

BAB II

PEMBAHASAN

2.1 Pengertian SLL.

Single Linked List (SLL) adalah struktur data linier yang terdiri dari kumpulan node atau simpul, dimana setiap node memiliki dua bagian utama yaitu data dan pointer (referensi) ke node berikutnya. Node pertama disebut sebagai head, sedangkan node terakhir memiliki pointer bernilai null yang menandakan akhir dari linked list.

Single Linked List memiliki keunggulan dalam fleksibilitas ukuran, karena tidak membutuhkan alokasi memori secara tetap seperti array. Penambahan dan penghapusan data dapat dilakukan dengan lebih mudah karena hanya perlu mengubah referensi node yang terhubung. Namun, kelemahan Single Linked List adalah proses pencarian data membutuhkan traversal dari awal hingga node yang dituju karena tidak memiliki akses indeks seperti array.

Dalam implementasi program Java yang diberikan, Single Linked List digunakan untuk melakukan beberapa operasi utama yaitu penambahan node, penghapusan node, dan pencarian node.

2.2 Struktur Node dalam SLL.

Dalam Single Linked List, node merupakan elemen dasar pembentuk linked list. Node menyimpan nilai data dan referensi ke node berikutnya. Pada kode `NodeSLL_2511533024.java`, node dibuat dalam bentuk class `NodeSLL_2511533024`.

Di dalam class tersebut terdapat dua atribut utama yaitu `data_3024` yang berfungsi untuk menyimpan nilai integer, serta `next_3024` yang berfungsi sebagai pointer atau referensi menuju node berikutnya. Selain itu, terdapat constructor `NodeSLL_2511533024(int data)` yang digunakan untuk menginisialisasi node baru. Constructor ini mengatur nilai `data_3024` sesuai parameter yang diberikan, dan mengatur `next_3024` menjadi null, yang berarti node tersebut belum terhubung dengan node lain.

Dengan adanya class node ini, maka program dapat membuat node-node baru yang kemudian dihubungkan satu sama lain sehingga membentuk sebuah linked list. Struktur node ini merupakan pondasi utama dalam proses operasi penambahan, penghapusan, maupun pencarian data pada linked list.

2.3 Operasi penambahan Node (insert) pada SLL.

Operasi penambahan node pada Single Linked List bertujuan untuk memasukkan data baru ke dalam linked list. Pada kode `TambahSLL_2511533024.java`, terdapat tiga metode utama untuk menambah node yaitu penambahan di depan (insert at front), penambahan di belakang (insert at end), dan penambahan pada posisi tertentu (insert at position).

Metode `insertAtFront` digunakan untuk menambahkan node baru di bagian paling depan linked list. Cara kerjanya adalah membuat node baru menggunakan constructor `NodeSLL_2511533024`, kemudian node baru tersebut diarahkan ke head lama melalui `new_Node3024.next_3024 = head_3024`. Setelah itu node baru dijadikan sebagai head baru dan dikembalikan sebagai hasil. Proses ini efektif karena tidak membutuhkan traversal panjang, sehingga operasi ini memiliki kompleksitas $O(1)$.

Metode `insertAtEnd` digunakan untuk menambahkan node di bagian paling belakang linked list. Pertama-tama program membuat node baru. Jika head bernilai null, maka node baru langsung dijadikan head. Namun jika linked list sudah memiliki isi, program melakukan traversal menggunakan variabel `last` sampai menemukan node terakhir (node dengan `next_3024 == null`). Setelah node terakhir ditemukan, node terakhir tersebut dihubungkan dengan node baru melalui `last.next_3024 = new_Node3024`. Metode ini membutuhkan traversal sehingga kompleksitasnya adalah $O(n)$.

Selanjutnya, metode `insertAtPos` digunakan untuk menambahkan node pada posisi tertentu. Program memeriksa terlebih dahulu apakah posisi valid. Jika posisi adalah 1, maka node baru langsung dijadikan head baru. Jika posisi lebih besar dari 1, program melakukan traversal menggunakan variabel `temp` sampai mencapai node sebelum posisi yang ditentukan. Jika posisi melebihi panjang linked list, maka program menampilkan pesan bahwa posisi di luar jangkauan. Jika posisi valid, maka node baru dibuat dan pointer `next_3024` disusun ulang sehingga node baru masuk pada posisi yang diinginkan. Metode ini memiliki kompleksitas $O(n)$ karena membutuhkan traversal.

Selain metode `insert`, program juga memiliki metode `printList_3024` yang digunakan untuk menampilkan isi linked list. Metode ini berjalan dengan cara melakukan traversal dari head sampai null, lalu mencetak setiap nilai data dengan format `-->` sebagai penghubung antar node. Dengan adanya metode ini, hasil operasi penambahan node dapat terlihat secara jelas.

Pada bagian main program, linked list awal dibuat secara manual dengan node 2 -> 3 -> 5 -> 6. Setelah itu dilakukan penambahan node di depan dengan nilai 1, penambahan node di belakang dengan nilai 7, serta penambahan node pada posisi ke-4 dengan nilai 4. Hasil akhirnya menunjukkan bahwa operasi penambahan node berjalan sesuai dengan konsep Single Linked List.

2.4 Operasi penghapusan Node (delete) pada SLL

Operasi penghapusan node bertujuan untuk menghilangkan node tertentu dari linked list sehingga ukuran linked list menjadi berkurang. Pada kode hapusSLL_2511533024.java, terdapat beberapa metode penghapusan yaitu menghapus node head, menghapus node terakhir, dan menghapus node pada posisi tertentu.

Metode deleteHead_3024 digunakan untuk menghapus node pertama atau head. Program memeriksa terlebih dahulu apakah head bernilai null. Jika null, berarti linked list kosong sehingga tidak ada yang bisa dihapus. Jika tidak kosong, maka head diarahkan ke node berikutnya dengan perintah `head_3024 = head_3024.next_3024`. Dengan demikian, node head lama akan terhapus karena tidak lagi direferensikan. Operasi ini sangat efisien karena kompleksitasnya $O(1)$.

Metode removeLastNode digunakan untuk menghapus node terakhir. Program memeriksa apakah linked list kosong. Jika kosong maka return null. Jika linked list hanya berisi satu node, maka node tersebut akan dihapus dengan mengembalikan null karena tidak ada node yang tersisa. Jika linked list memiliki lebih dari satu node, maka program menggunakan variabel secondLast untuk mencari node kedua terakhir. Hal ini dilakukan dengan traversal sampai kondisi `secondLast.next_3024.next_3024 != null`. Setelah node kedua terakhir ditemukan, node terakhir dihapus dengan cara mengubah pointer `secondLast.next_3024 = null`. Dengan cara ini node terakhir tidak lagi terhubung sehingga dianggap terhapus. Kompleksitas metode ini adalah $O(n)$ karena membutuhkan traversal.

Metode deleteNode digunakan untuk menghapus node pada posisi tertentu. Program menggunakan dua variabel yaitu temp sebagai node yang sedang dicek, dan prev sebagai node sebelumnya. Jika posisi yang diminta adalah 1, maka proses sama seperti delete head, yaitu head dipindahkan ke node berikutnya. Namun jika posisi lebih besar dari 1, program melakukan traversal menggunakan loop for sampai mencapai posisi yang dimaksud. Setelah node ditemukan, pointer node sebelumnya diarahkan ke node setelah node yang akan dihapus menggunakan `prev.next_3024 = temp.next_3024`. Dengan demikian node pada posisi tersebut terhapus dari linked list. Jika node pada posisi yang dimaksud tidak ditemukan, maka program mencetak pesan "data tidak ada". Metode ini juga memiliki kompleksitas $O(n)$.

Selain operasi delete, terdapat metode printList_3024 yang berfungsi menampilkan isi linked list. Metode ini melakukan traversal dari head dan mencetak data node dengan format `-->`. Pada main program, linked list dibuat dengan nilai `1 -> 2 -> 3 -> 4 -> 5 -> 6`, kemudian dilakukan penghapusan head, penghapusan node terakhir, serta penghapusan node pada posisi 2. Dari hasil program terlihat bahwa operasi delete berjalan dengan benar sesuai konsep Single Linked List.

2.5 Operasi pencarian data Node (search) pada SLL

Operasi pencarian digunakan untuk menemukan apakah suatu nilai tertentu terdapat dalam linked list atau tidak. Pada kode pencarianSLL_2511533024.java, terdapat metode searchKey yang digunakan untuk mencari data dalam linked list.

Metode searchKey bekerja dengan cara traversal dari head hingga akhir linked list. Program menggunakan variabel curr sebagai node yang sedang diperiksa. Selama curr tidak bernilai null, program membandingkan nilai curr.data_3024 dengan nilai key yang dicari. Jika nilai tersebut sama, maka metode langsung mengembalikan true yang berarti data ditemukan. Jika tidak ditemukan sampai akhir traversal, maka metode mengembalikan false.

Selain itu terdapat metode traversal yang digunakan untuk menampilkan isi linked list sebelum dilakukan pencarian. Metode traversal ini juga melakukan iterasi dari head hingga null dan mencetak data setiap node.

Pada bagian main program, linked list dibuat dengan nilai 14 -> 21 -> 13 -> 30 -> 10. Kemudian dilakukan pencarian terhadap key 30. Program menampilkan isi linked list terlebih dahulu, lalu menampilkan hasil pencarian apakah data tersebut "ketemu" atau "tidak ada". Berdasarkan hasil program, nilai 30 berhasil ditemukan sehingga program menampilkan output "ketemu". Metode pencarian ini memiliki kompleksitas $O(n)$ karena membutuhkan traversal dari awal linked list.

2.6 Analisis kompleksitas waktu SLL.

Berdasarkan implementasi program yang diberikan, kompleksitas waktu untuk setiap operasi pada Single Linked List dapat dijelaskan sebagai berikut. Operasi penambahan node di depan memiliki kompleksitas $O(1)$ karena tidak membutuhkan traversal. Operasi penambahan node di belakang memiliki kompleksitas $O(n)$ karena harus mencari node terakhir terlebih dahulu. Operasi penambahan node pada posisi tertentu juga memiliki kompleksitas $O(n)$ karena harus melakukan traversal hingga posisi tertentu.

Pada operasi penghapusan node, penghapusan head memiliki kompleksitas $O(1)$ karena cukup memindahkan pointer head. Penghapusan node terakhir memiliki kompleksitas $O(n)$ karena harus mencari node kedua terakhir. Penghapusan node pada posisi tertentu juga memiliki kompleksitas $O(n)$ karena membutuhkan traversal sampai posisi yang ditentukan.

Pada operasi pencarian data, kompleksitasnya adalah $O(n)$ karena harus memeriksa node satu per satu sampai menemukan data atau sampai akhir linked list. Dari analisis tersebut dapat disimpulkan bahwa Single Linked List sangat efisien untuk operasi penambahan atau penghapusan di bagian depan, namun kurang efisien untuk operasi yang membutuhkan pencarian node tertentu karena harus melakukan traversal.

BAB III

KESIMPULAN

Berdasarkan pembahasan yang telah dijelaskan, dapat disimpulkan bahwa Single Linked List merupakan struktur data dinamis yang terdiri dari node-node yang saling terhubung melalui pointer atau referensi. Single Linked List memiliki keunggulan dalam fleksibilitas ukuran karena dapat bertambah atau berkurang sesuai kebutuhan program.

Implementasi Single Linked List menggunakan Java dilakukan dengan membuat class node yang berisi data dan pointer next. Operasi penambahan node dapat dilakukan di bagian depan, belakang, maupun pada posisi tertentu dengan cara mengatur ulang referensi antar node. Operasi penghapusan node juga dapat dilakukan pada head, tail, maupun posisi tertentu dengan cara memutus hubungan node yang akan dihapus dari linked list. Selain itu, pencarian data dilakukan dengan traversal dari head sampai akhir linked list dan membandingkan setiap nilai data dengan key yang dicari.

Single Linked List memiliki efisiensi yang baik untuk operasi insert dan delete pada bagian depan dengan kompleksitas $O(1)$, namun untuk operasi pencarian dan insert/delete di bagian tengah atau akhir membutuhkan traversal sehingga kompleksitasnya menjadi $O(n)$.

DAFTAR PUSTAKA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd Edition). MIT Press.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th Edition). Wiley.
- Weiss, M. A. (2012). *Data Structures and Algorithm Analysis in Java* (3rd Edition). Pearson.
- Oracle. (2024). *Java Documentation*. Oracle Corporation.
- Modul Mata Kuliah Struktur Data (Single Linked List), Materi Perkuliahan.